SİVAS CUMHURİYET ÜNİVERSİTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

GÖMÜLÜ SİSTEMLER DERSİ

Gömülü Sistemler Deney 4: Zamanlayıcı (Timer) ve Kesme (Interrupt) Uygulamaları

Akıllı Geri Sayım Sayacı, Röle Kontrolü ve Görsel/İşitsel Alarm Sistemi

Dersin Hocası: Doç. Dr. Ahmet Gürkan YÜKSEK
Dersin Asistanı: Arş. Gör. Ahmet Utku ELİK
Öğrenci Adı Soyadı – Numarası:

1. DENEYİN AMACI

Bu deneyin amacı, gömülü sistemlerde zamanlama işlemlerinin **Polling** (Sürekli Sorgulama) yöntemi yerine, donanımsal **Timer (Zamanlayıcı)** birimi ve **Interrupt (Kesme)** mekanizması kullanılarak gerçekleştirilmesini öğrenmektir.

Deney kapsamında, ArmApp-18 eğitim seti üzerinde bir "Geri Sayım Sayacı" tasarlanacaktır. Zamanlayıcıdan alınan 1 saniyelik periyodik kesmeler ile süre işletilecek, süre dolduğunda ise **Röle (Relay)** tetiklenerek yüksek güçlü bir yükün anahtarlanması simüle edilecektir. Ayrıca I2C LCD ekran ve Buzzer kullanılarak sisteme Görsel/İşitsel geri bildirim yeteneği kazandırılması hedeflenmektedir.

2. TEORİK BİLGİ

2.1. Timer (Zamanlayıcı) ve Kesme Nedir? Mikrodenetleyici içerisinde bulunan Timer birimi, ana işlemciden (CPU) bağımsız olarak çalışan ve sistem saat sinyallerini sayan bir donanımdır.

- **Polling Yöntemi:** İşlemci sürekli saati kontrol eder ("Süre doldu mu?"). Bu durum işlemciyi meşgul eder ve verimsizdir.
- **Interrupt (Kesme) Yöntemi:** Timer donanımı süreyi arka planda sayar. Süre dolduğunda işlemciye "Dur!" sinyali gönderir. İşlemci o anki işini bırakır, ilgili fonksiyonu çalıştırır ve kaldığı yerden devam eder.

2.2. Röle (Relay) Çalışma Prensibi Röle, düşük akımlı bir kontrol sinyali (mikrodenetleyici çıkışı) ile yüksek akımlı bir devreyi (Örn: 220V Motor, Lamba) kontrol etmeye yarayan elektromekanik bir anahtardır.

- **Mantık 1 (High):** Röle bobini enerjilenir, elektromıknatıs kontağı çeker ("ÇIT" sesi) ve devre tamamlanır.
- **Mantık 0 (Low):** Enerji kesilir, yay mekanizması kontağı açar ve devre kesilir.

2.3. Zamanlama Hesabı (Matematiksel Model) Deneyde sistemin tam olarak **1 Saniye (1 Hz)** aralıklarla çalışması istenmektedir. STM32 Timer frekansı aşağıdaki formülle hesaplanır:

$$\text{Time} = \frac{(PSC + 1) * (ARR + 1)}{Fclk}$$

- **Fclk (Saat Frekansı):** 16 MHz (16.000.000 Hz)
- **PSC (Prescaler):** 15999 (Saati 16000'e böler -> 1 kHz sayaç hızı)
- **ARR (Period):** 999 (1000 adım sayar ->1000 ms = 1 Saniye)

3. KULLANILACAK DONANIM VE PIN TABLOSU

Bu deneyde, **ArmApp-18** Gömülü Sistemler Eğitim Seti üzerinde bulunan bileşenler kullanılacaktır. Harici bir devre kurulumu yapılmayacak, set üzerindeki dahili bağlantılar aktif edilecektir.

Kullanılan Malzemeler:

1. STM32F407 Discovery Geliştirme Kartı (ArmApp-18 Setine Entegre)
2. ArmApp-18 Deney Seti (LCD, Röle, Buzzer ve LED Modülleri)
3. USB Bağlantı Kablosu (Mini-USB)

Pin Bağlantı Tablosu: Aşağıdaki tabloda, deneyde kullanılacak çevresel birimlerin mikrodenetleyici pin karşılıkları ve CubeMX modları belirtilmiştir.

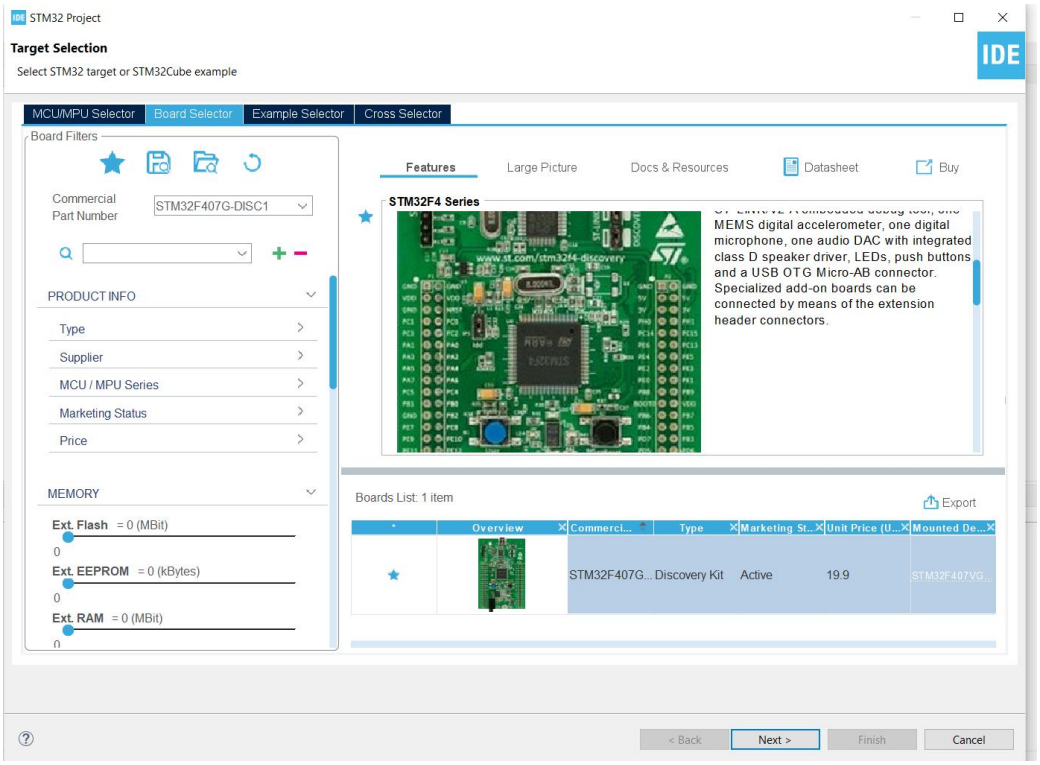
Bileşen	Pin Kodu	Mod (Mode)	Açıklama
LCD RS	PD2	GPIO_Output	LCD Komut/Veri Seçimi
LCD EN	PD3	GPIO_Output	LCD Enable Sinyali
LCD D4	PC4	GPIO_Output	LCD Veri Hattı 4
LCD D5	PC13	GPIO_Output	LCD Veri Hattı 5
LCD D6	PC14	GPIO_Output	LCD Veri Hattı 6
LCD D7	PC15	GPIO_Output	LCD Veri Hattı 7
Set LED 0	PE0	GPIO_Output	Durum LED'i
Set LED 1	PE1	GPIO_Output	Alarm LED'i
Buzzer	PA8	GPIO_Output	Sesli Alarm Çıkışı
Röle (Relay)	PC7	GPIO_Output	Yük Kontrol Anahtarı
Timer 2	-	Internal Clock	1 Hz (1 Saniye) Zamanlayıcı

4. STM32CubeIDE KONFİGÜRASYONU (Donanım Ayarları)

Kodlama aşamasına geçmeden önce, mikrodenetleyicinin çevresel birimlerinin (GPIO, Timer, NVIC) grafiksel arayüz (CubeMX) üzerinden yapılandırılması gerekmektedir.

Adım 4.1: Proje Oluşturma

1. **STM32CubeIDE** programını başlatın.
2. Üst menüden **File > New > STM32 Project** yolunu izleyin.
3. Açılan "Target Selection" penceresindeki **Part Number** arama çubuğuna, deney setinde kullanılan işlemci modelini (**STM32F407VG**) yazın.



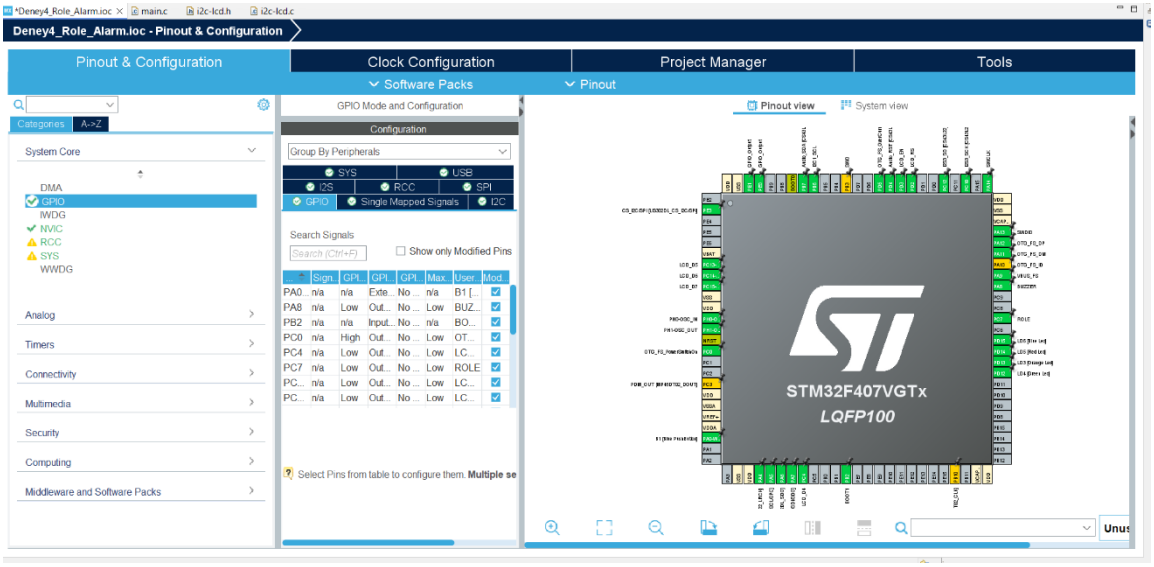
1. Sağdaki listeden işlemciyi seçip **Next** butonuna basın.
2. Proje ismine **"Deney4_Role_Alarm"** yazıp **Finish** butonuna tıklayın.

Adım 4.2: Pin Atamaları (GPIO Konfigürasyonu)

ArmApp-18 eğitim seti üzerindeki bileşenlerin (LCD, Röle, Buzzer, LED) işlemci ile haberleşebilmesi için ilgili pinlerin "Çıkış" (Output) olarak ayarlanması gerekir.

Pinout & Configuration ekranındaki işlemci görseli üzerinde aşağıdaki pinleri tek tek bulun ve yapılandırın:

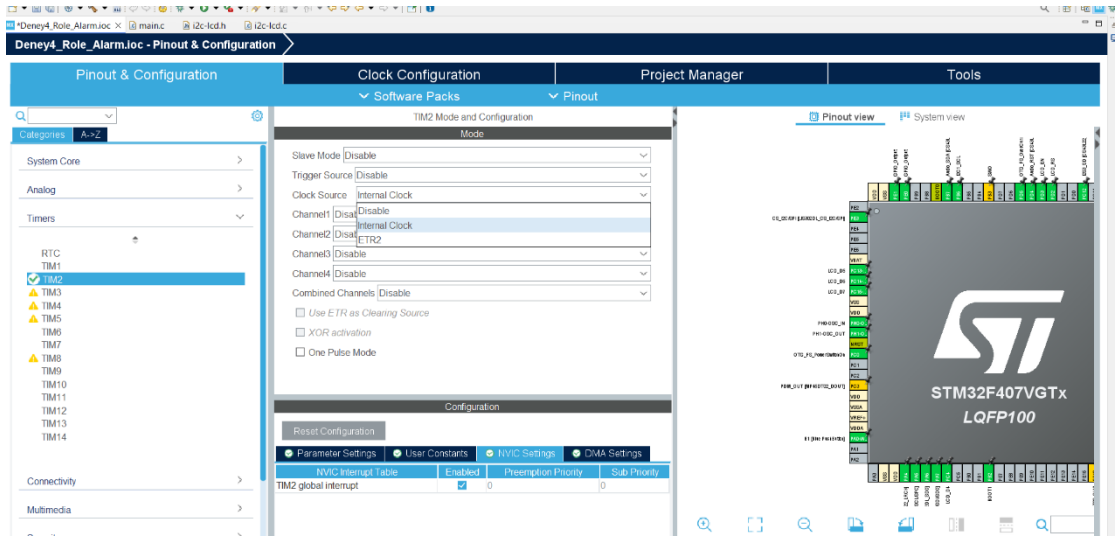
1. **LCD Ekran Pinleri (Paralel Bağlantı):**
   - Aşağıdaki pinlerin her birine **Sol Tıklayın** ve açılan listeden **GPIO_Output** seçeneğini işaretleyin:
     - **PD2** (RS) ve **PD3** (EN)
     - **PC4** (D4), **PC13** (D5), **PC14** (D6), **PC15** (D7)
   - *Öneri:* Pinlere sağ tıklayıp **Enter User Label** diyerek şematikteki isimlerini (Örn: LCD_RS, LCD_D4) yazmanız kod yazarken kolaylık sağlar.
2. **Sistem Kontrol Pinleri:**
   - **PC7:** Röle kontrolü için **GPIO_Output** olarak ayarlayın. (Etiket: ROLE)
   - **PA8:** Buzzer kontrolü için **GPIO_Output** olarak ayarlayın. (Etiket: BUZZER)
   - **PE0** ve **PE1:** Durum LED'leri için **GPIO_Output** olarak ayarlayın.



Adım 4.3: Zamanlayıcı (Timer 2) Ayarları

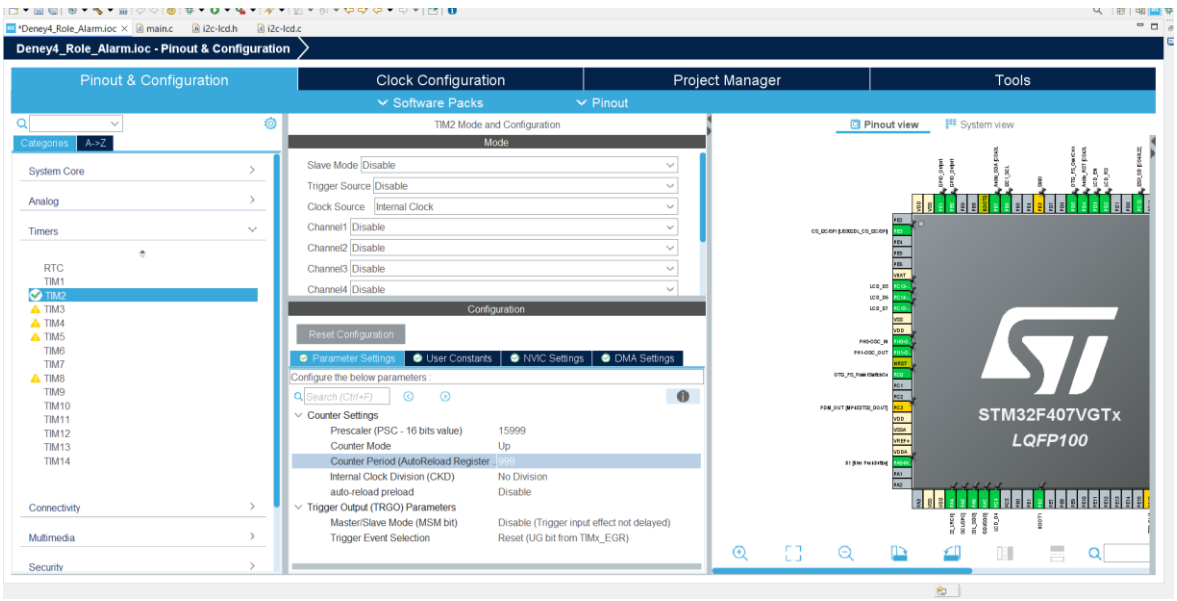
Sistemin tam olarak **1 Saniye (1 Hz)** aralıklarla kesme üretmesi için TIM2 birimi aşağıdaki gibi ayarlanmalıdır.

1. Sol taraftaki kategoriler menüsünden **Timers** başlığını genişletin ve **TIM2** seçeneğine tıklayın.
2. **TIM2 Mode and Configuration** panelinde **Clock Source** seçeneğini **Internal Clock** olarak değiştirin.



3. Alt kısımdaki **Configuration > Parameter Settings** sekmesine şu değerleri girin:

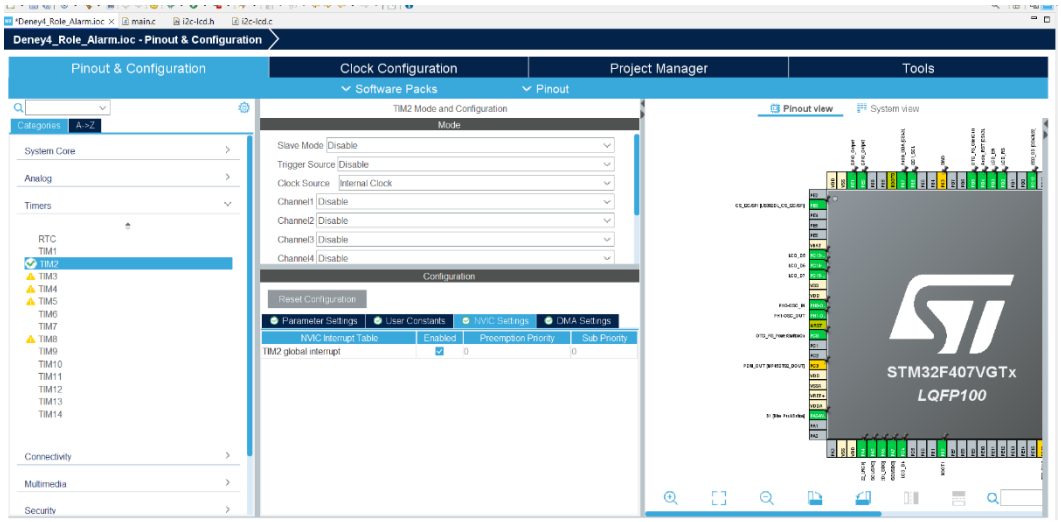
- **Prescaler (PSC - 16 bit value):** 15999
- **Counter Period (ARR - 32 bit value):** 999
- *Hesaplama:* (16 MHz / (15999+1)) = 1000 Hz (Sayaç Frekansı). 1000 Hz ile 1000 adım (0-999) saymak tam 1 saniye sürer.



Adım 4.4: Kesme (Interrupt) Aktivasyonu

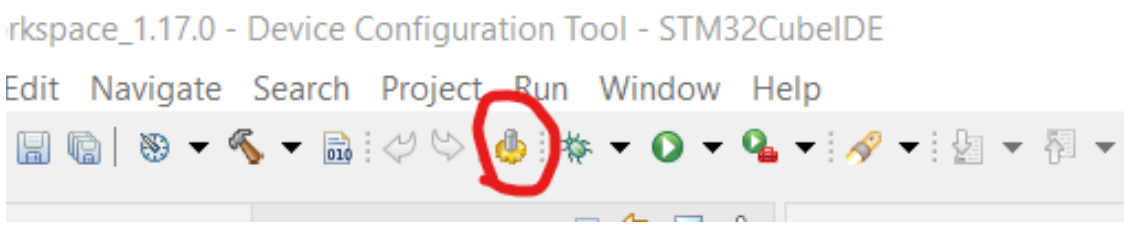
Timer süresi dolduğunda (Update Event), işlemcinin ana programı durdurup bizim yazdığımız fonksiyona gitmesi için kesme izni verilmelidir.

1. TIM2 konfigürasyon penceresinde **NVIC Settings** sekmesine geçin.
2. Listede **TIM2 global interrupt** satırını bulun.
3. En sağdaki **Enabled** kutucuğuna tıklayarak **Tik (✓)** işaretini koyun.



Adım 4.5: Kod Üretimi

Yapılan ayarların C koduna dönüştürülmesi için üst menüdeki **"Device Configuration Tool Code Generation"** (Sarı Dişli Çark) ikonuna tıklayın ve gelen uyarıya "Yes" diyerek main.c dosyasını oluşturun.



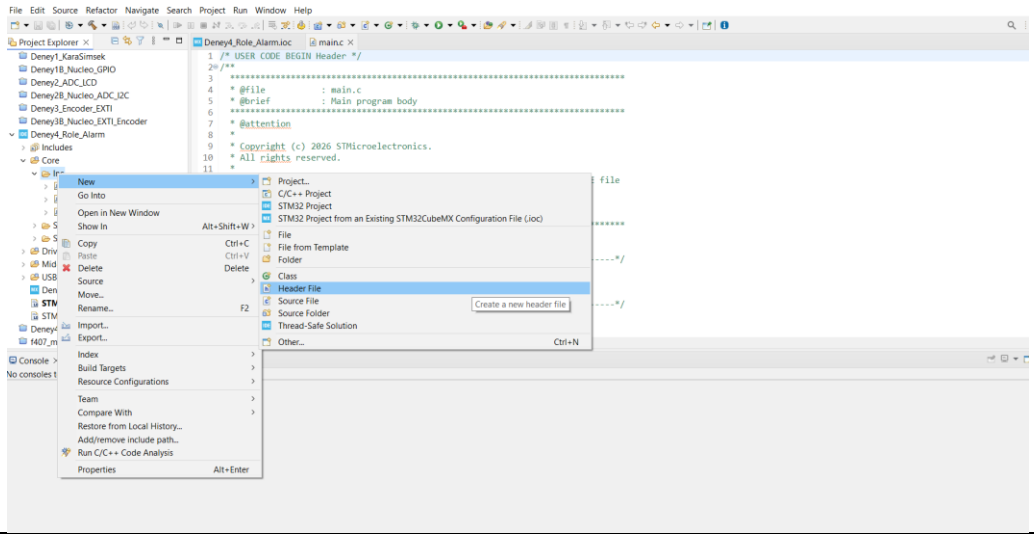
5. YAZILIM GELİŞTİRME

STM32CubeIDE tarafından oluşturulan standart HAL kütüphanesi içerisinde karakter LCD ekranlar için hazır bir sürücü bulunmamaktadır. Bu nedenle, LCD ekranı kontrol etmek (başlatmak, yazı yazmak, temizlemek) için lcd.h ve lcd.c dosyaları manuel olarak oluşturulup projeye dahil edilecektir.

Adım 5.1: LCD Kütüphane Dosyalarının Oluşturulması

1. lcd.h (Başlık Dosyası) Oluşturma: Bu dosya, LCD ekranın hangi pinlere bağlı olduğunu tanımlar ve kullanılacak fonksiyonların listesini içerir.

- **Project Explorer** penceresinde **Core > Inc** klasörüne sağ tıklayın.
- **New > Header File** seçeneğini tıklayın.



- File name kısmına: **lcd.h** yaz ve **Finish** de.

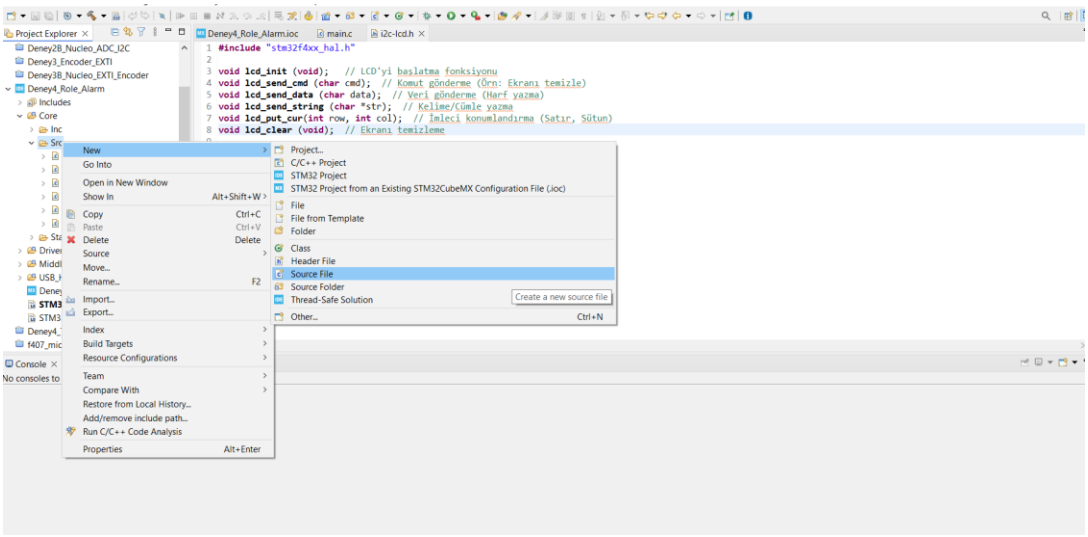
- Açılan boş sayfaya aşağıdaki kodu ekleyin.

```
#ifndef LCD_H_
#define LCD_H_

#include "stm32f4xx_hal.h"

/* --- ArmApp-18 LCD Pin Tanımlamaları --- */
// RS ve EN Pinleri (Kontrol)
#define RS_PORT GPIOD
#define RS_PIN GPIO_PIN_2

#define EN_PORT GPIOD
#define EN_PIN GPIO_PIN_3

// Veri (Data) Pinleri - 4 Bit Mod
#define D4_PORT GPIOC
#define D4_PIN GPIO_PIN_4

#define D5_PORT GPIOC
#define D5_PIN GPIO_PIN_13

#define D6_PORT GPIOC
#define D6_PIN GPIO_PIN_14

#define D7_PORT GPIOC
#define D7_PIN GPIO_PIN_15

/* --- Fonksiyon Prototipleri --- */
void Lcd_Init(void);                      // Ekranı başlatır
void Lcd_Cmd(uint8_t cmd);                // Komut gönderir (Temizle, kaydır vb.)
void Lcd_Char(char data);                 // Tek karakter yazar
void Lcd_String(char *str);               // Kelime/Cümle yazar
void Lcd_Set_Cursor(uint8_t row, uint8_t col); // İmleci konumlandırır (Satır,
Sütun)
void Lcd_Clear(void);                     // Ekranı temizler

#endif
```

- **CTRL+S** yaparak kaydet ve dosyayı kapat.

2. lcd.c (Kaynak Dosyası) Oluşturma: Bu dosya, lcd.h dosyasında tanımlanan fonksiyonların gerçek çalışma kodlarını içerir.

- **Project Explorer** penceresinde **Core > Src** klasörüne sağ tıklayın.
- **New > Source File** seçeneğini tıklayın.



- File name kısmına: **lcd.c** yaz ve **Finish** de.
- Açılan boş sayfaya aşağıdaki kodu ekleyin

```
#include "lcd.h"

// Yardımcı Fonksiyon: 4-Bit veriyi ilgili pinlere dağıtır
void Lcd_Port(uint8_t data)
{
	HAL_GPIO_WritePin(D4_PORT, D4_PIN, (data & 0x01) ? GPIO_PIN_SET :
GPIO_PIN_RESET);
	HAL_GPIO_WritePin(D5_PORT, D5_PIN, (data & 0x02) ? GPIO_PIN_SET :
GPIO_PIN_RESET);
	HAL_GPIO_WritePin(D6_PORT, D6_PIN, (data & 0x04) ? GPIO_PIN_SET :
GPIO_PIN_RESET);
	HAL_GPIO_WritePin(D7_PORT, D7_PIN, (data & 0x08) ? GPIO_PIN_SET :
GPIO_PIN_RESET);
}

// LCD'ye Komut Gönderme (RS=0)
void Lcd_Cmd(uint8_t cmd)
{
	HAL_GPIO_WritePin(RS_PORT, RS_PIN, GPIO_PIN_RESET); // Komut Modu

	Lcd_Port(cmd >> 4); // Üst 4 bit gönder
	HAL_GPIO_WritePin(EN_PORT, EN_PIN, GPIO_PIN_SET);
	HAL_Delay(1);
	HAL_GPIO_WritePin(EN_PORT, EN_PIN, GPIO_PIN_RESET);
	HAL_Delay(1);

	Lcd_Port(cmd); // Alt 4 bit gönder
	HAL_GPIO_WritePin(EN_PORT, EN_PIN, GPIO_PIN_SET);
	HAL_Delay(1);
	HAL_GPIO_WritePin(EN_PORT, EN_PIN, GPIO_PIN_RESET);
	HAL_Delay(2);
}

// LCD'ye Karakter Yazma (RS=1)
void Lcd_Char(char data)
{
	HAL_GPIO_WritePin(RS_PORT, RS_PIN, GPIO_PIN_SET); // Veri Modu

	Lcd_Port(data >> 4);
	HAL_GPIO_WritePin(EN_PORT, EN_PIN, GPIO_PIN_SET);
	HAL_Delay(1);
	HAL_GPIO_WritePin(EN_PORT, EN_PIN, GPIO_PIN_RESET);
	HAL_Delay(1);

	Lcd_Port(data);
	HAL_GPIO_WritePin(EN_PORT, EN_PIN, GPIO_PIN_SET);
	HAL_Delay(1);
	HAL_GPIO_WritePin(EN_PORT, EN_PIN, GPIO_PIN_RESET);
```

```
	HAL_Delay(2);
}

// Metin Yazma Fonksiyonu
void Lcd_String(char *str)
{
	while(*str) Lcd_Char(*str++);
}

// İmleç Konumlandırma (0. veya 1. Satır)
void Lcd_Set_Cursor(uint8_t row, uint8_t col)
{
    uint8_t mask = (row == 0) ? 0x80 : 0xC0;
    Lcd_Cmd(mask | col);
}

// Ekran Temizleme
void Lcd_Clear(void)
{
	Lcd_Cmd(0x01);
	HAL_Delay(2);
}

// Başlangıç Ayarları (Initialization)
void Lcd_Init(void)
{
	HAL_Delay(50); // Ekranın elektriksel olarak oturmasını bekle
	Lcd_Cmd(0x02); // 4-Bit moda geçiş
	Lcd_Cmd(0x28); // 2 Satır, 5x8 Font
	Lcd_Cmd(0x0C); // Ekran Açık, İmleç Kapalı
	Lcd_Cmd(0x06); // Yazdıkça imleci sağa kaydır
	Lcd_Clear();
}
```

Adım 5.2: main.c Kodlaması

Oluşturulan kütüphane ve Timer kesmesi main.c dosyasına entegre edilerek ana algoritma kurulur.

1. Kütüphane ve Değişkenlerin Tanımlanması: main.c dosyasının en üst kısmındaki Includes ve PV (Private Variables) bloklarına aşağıdaki kodlar eklenir.

```c
/* USER CODE BEGIN Includes */
#include "lcd.h"   // LCD kütüphanesini çağır
#include <stdio.h> // "sprintf" fonksiyonu için gerekli
/* USER CODE END Includes */

/* USER CODE BEGIN PV */
volatile uint8_t timer_bayrak = 0; // Kesmeden ana döngüye haber taşıyan bayrak
int sayac = 10;                    // Geri sayım süresi (Saniye)
char msg[20];                      // LCD'ye yazılacak metin kutusu
/* USER CODE END PV */
```

2. Kesme Geri Çağırma (Callback) Fonksiyonu: Timer süresi dolduğunda (1 saniye geçtiğinde) işlemcinin otomatik olarak çalıştırdığı fonksiyondur. Bu fonksiyon main.c dosyasının en altına (/* USER CODE BEGIN 4 */ bloğuna) eklenir.

```c
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    // Eğer süresi dolan zamanlayıcı TIM2 ise
    if (htim->Instance == TIM2)
    {
        timer_bayrak = 1; // Ana döngüye "Süre Doldu" mesajı ver
    }
}
/* USER CODE END 4 */
```

3. Ana Program Döngüsü (Main Loop): Sistemin başlangıç ayarları ve geri sayım mantığı burada kurulur.

```
/* USER CODE BEGIN 2 */
  // --- SİSTEM BAŞLANGIÇ AYARLARI ---
  Lcd_Init();                    // LCD Ekranı Başlat
  Lcd_String("SISTEM HAZIR");    // Açılış Mesajı
  HAL_Delay(1000);               // 1 saniye bekle
  Lcd_Clear();                   // Ekranı temizle

  // Güvenlik: Başlangıçta Röle ve Buzzer KAPALI olsun
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_7, GPIO_PIN_RESET);
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);

  // Timer 2'yi Kesme (Interrupt) Modunda Başlat
  HAL_TIM_Base_Start_IT(&htim2);
  /* USER CODE END 2 */

  /* USER CODE BEGIN WHILE */
  while (1)
  {
      // --- ZAMANLAYICI KONTROLÜ ---
      // Kesme fonksiyonundan haber geldi mi? (1 Saniye doldu mu?)
      if (timer_bayrak == 1)
      {
          timer_bayrak = 0; // Haberi aldık, bayrağı indir (Resetle)

          // Eğer süre henüz bitmediyse (NORMAL MOD)
          if (sayac > 0)
          {
              sayac--; // Süreyi 1 azalt

              // LCD Ekranı Güncelle
              Lcd_Set_Cursor(0, 0);
              Lcd_String("GERI SAYIM...");

              Lcd_Set_Cursor(1, 0);
              sprintf(msg, "KALAN: %02d SN   ", sayac); // Sayıyı metne çevir
              Lcd_String(msg);

              // Sistem Çalışıyor Efekti: PE0 LED'ini yak/söndür
              HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_0);
          }
          // Süre bittiyse (ALARM MODU)
          else
          {
              // 1. RÖLEYİ TETİKLE (PC7 -> Logic 1)
              // Bu işlem, röleye bağlı motoru/lambayı çalıştırır.
              HAL_GPIO_WritePin(GPIOC, GPIO_PIN_7, GPIO_PIN_SET);
```

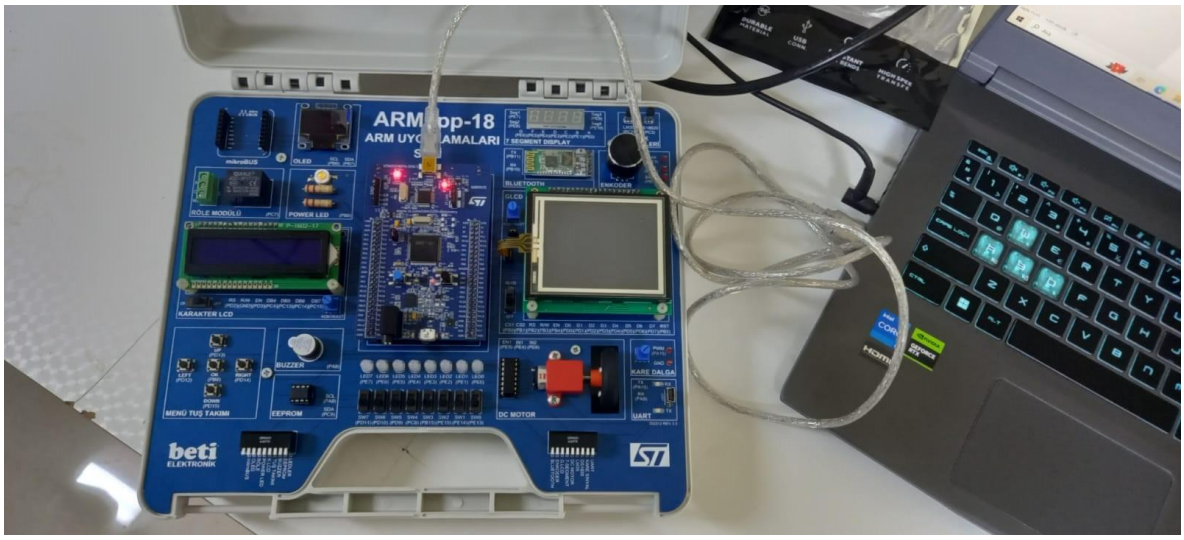
```
            // 2. LCD'ye Alarm Mesajı Yaz
            Lcd_Set_Cursor(0, 0);
            Lcd_String("!! ALARM !!   ");
            Lcd_Set_Cursor(1, 0);
            Lcd_String("MOTOR AKTIF   ");

            // 3. Buzzer ve Alarm LED'ini (PE1) Flaşör Yap
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_8); // Buzzer öter/susar
            HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1); // Kırmızı LED yanıp söner
        }
    }
  /* USER CODE END WHILE */
```

6. Pratik Uygulama ve Test Adımları

Yazdığımız kodun STM32 kartına yüklenmesi ve deney seti üzerinde test edilmesi için aşağıdaki adımları izleyin.

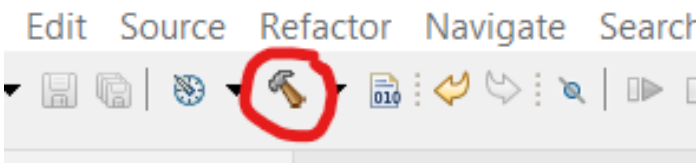
Adım 1: Fiziksel Bağlantı

1. ArmApp18 deney seti üzerindeki **STM32F4 Discovery** kartının üst kısmında bulunan **mini-USB (USB ST-LINK)** girişine kabloyu takın.
2. Kablonun diğer ucunu bilgisayarınızın USB portuna bağlayın.
3. Kart üzerindeki **PWR** (Güç) LED'inin yandığını ve bilgisayarınızın donanımı tanıdığını doğrulayın.

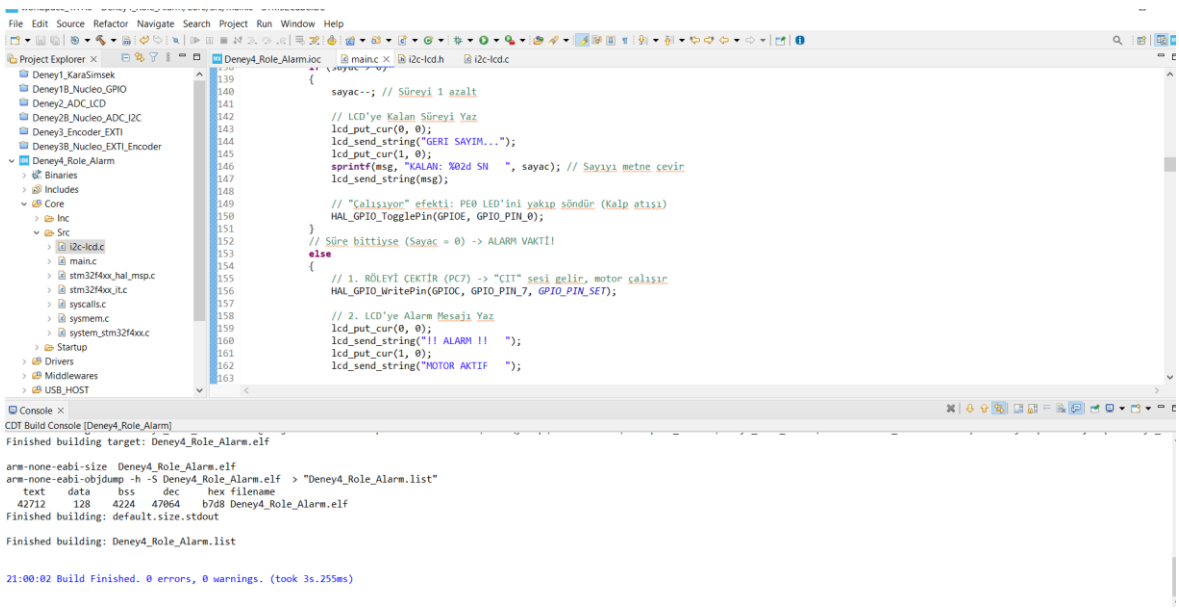


Adım 2: Derleme ve Yükleme (Build & Run)

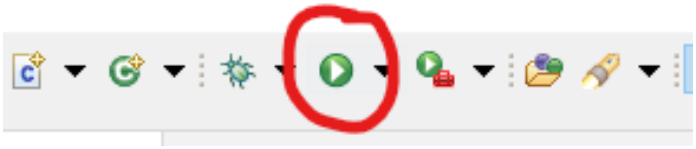
1. **Derleme:** STM32CubeIDE üst menüsündeki **"Çekiç" (Build)** ikonuna tıklayın (veya Ctrl+B).



- *Console penceresinde "0 errors, 0 warnings" mesajını görmelisiniz.*



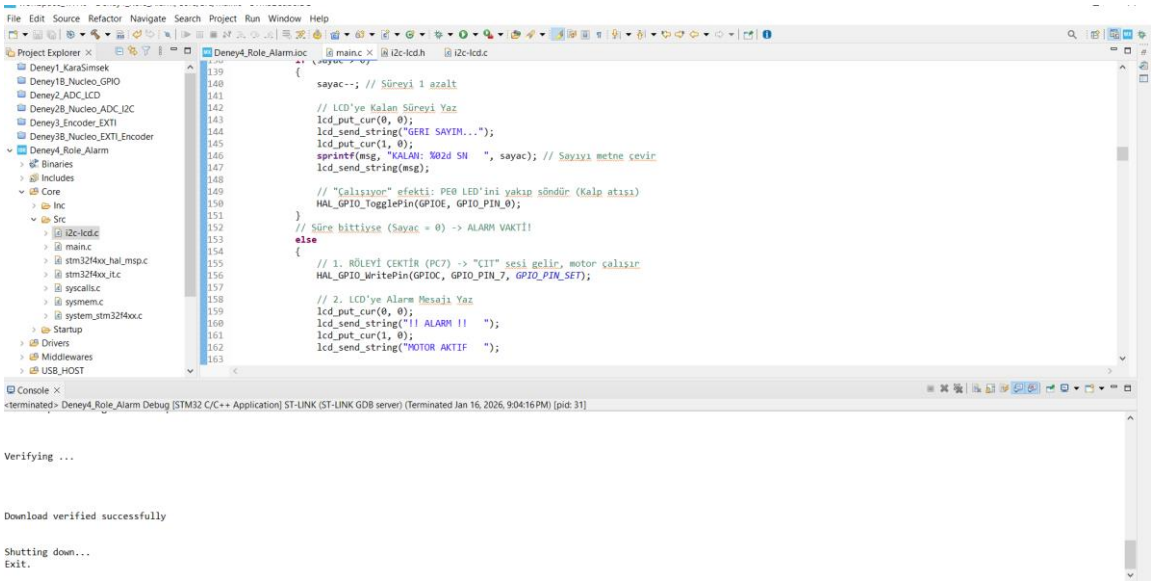
2. **Yükleme:** Derleme hatasız ise, yeşil **"Oynat" (Run)** ikonuna tıklayın.



- *İlk yüklemede "Edit Configuration" penceresi açılırsa "OK" diyerek geçin.*

- *ST-LINK Firmware uyarısı gelirse "Upgrade" işlemini yapın.*

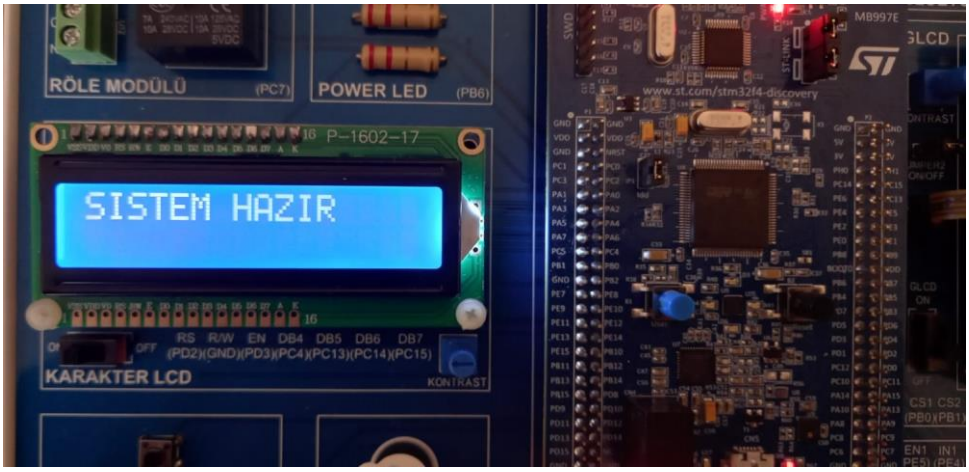
3. **Onay:** Console penceresinde **"Download verified successfully"** mesajını gördüğünüzde kod karta yüklenmiş demektir.



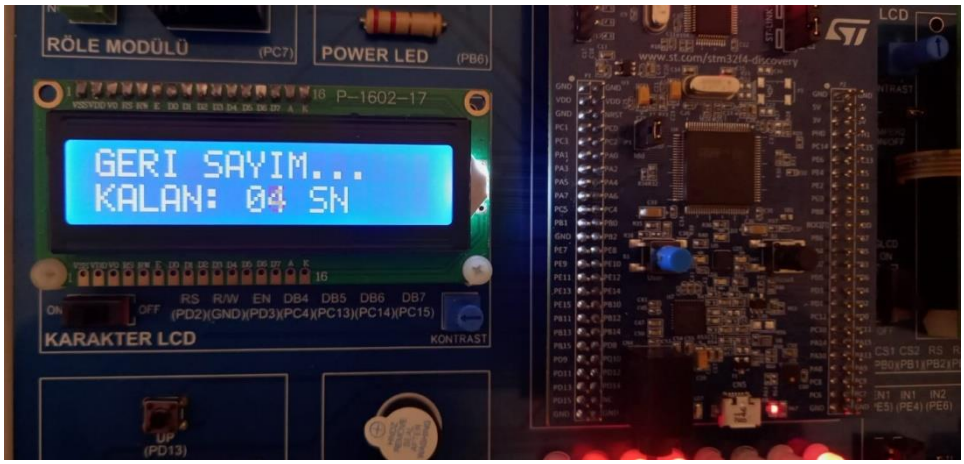
7. DENEY SONUÇLARI VE GÖZLEMLER

Yapılan kodlama ve donanım konfigürasyonu sonucunda ArmApp-18 eğitim seti üzerinde aşağıdaki davranışlar gözlemlenmiştir:

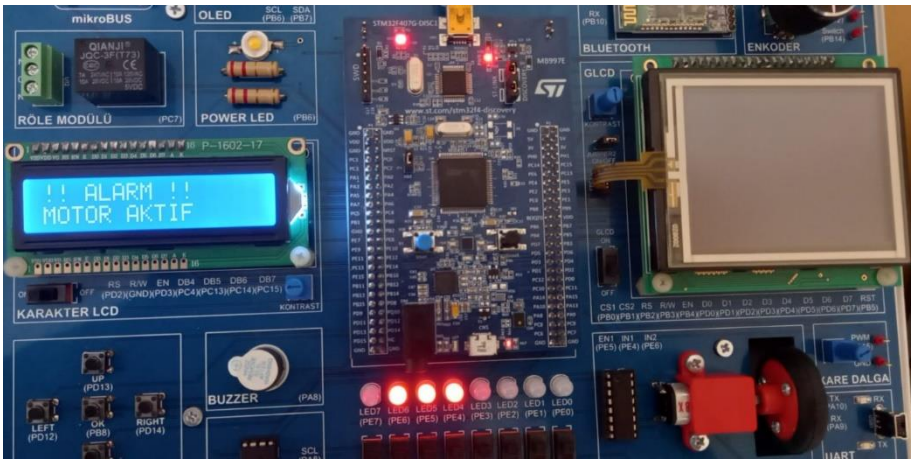
1. **Sistem Açılışı:** Enerji verildiğinde LCD başlatılmış, ekranda 1 saniye boyunca **"SISTEM HAZIR"** mesajı görülmüştür. Röle ve Buzzer güvenli (Pasif) durumda kalmıştır.



2. **Geri Sayım Süreci:** Timer 2 biriminden her 1 saniyede bir gelen kesme sinyali ile;
    - LCD ekrandaki sayaç değeri (10, 09, 08...) düzenli olarak azalmıştır.



    - Durum LED'i (PE0) her saniye yanıp sönerek sistemin çalıştığını (Heartbeat) görselleştirmiştir.
3. **Alarm Durumu:** Süre **00** olduğunda sistem Alarm Moduna geçmiş;
    - **Röle (PC7)** tetiklenmiş ve modül üzerinden kontak çekme sesi ("ÇIT") duyulmuştur. Röle LED'i aktif olmuştur.
    - LCD ekranda **"!! ALARM !!"** ve **"MOTOR AKTIF"** uyarısı görüntülenmiştir.



    - Buzzer ve Kırmızı LED (PE1) eş zamanlı olarak uyarı vermiştir.